Distributed Message Queue
Kafka vs Rabbitmq
Kafka Consumer Rebalancing
Kafka Broker Leader Election
Kafka Transaction
Kafka Exactly Once Semantics
S3 Object Storage
Redis Cache Invalidation
Redis Sorted Set
Redis Cluster
Redis Pub/Sub
Redis Lua Scripting
Redis Pipelining
Redis Sentinel
Redis Cluster Gossip Protocol
Redis Cluster Hash Slot
Redis Cluster Failover
Redis Cluster Resharding
Redis Cluster Client Routing
Redis Cluster Scaling
Redis Cluster Limitations
Redis Cluster Multi-Key Operations
Redis Cluster Pub/Sub
Redis Cluster Transactions
Redis Cluster Lua Scripting
Redis Cluster Pipelining
Redis Cluster

Object lookup

With each file having many small objects, then there the data node locate an object by OUOTP. The data node needs the following information:

- The data file that contains the object
- The starting offset of the object in the data file
- The size of the object

The database schema to support this lookup is shown in Table 1.

object_mapping
object_id
file_name
start_offset
object_size

Table 1. Object mapping table

Field	Description
object_id	UUID of the object
file_name	The name of the file that contains the object
start_offset	Beginning address of the object in the file
object_size	The number of bytes in the object

Table 2. Object_mapping fields

We combined the options for storing the mapping in the based key-value store such as RocksDB [16] or a relational database. RocksDB provides better write latency, but it is harder to scale than a relational database. Nonetheless, we decided to use RocksDB for the mapping service, and it is fast for reads and writes for writes. Its incremental backup, the ability to write across years and make multiple copies, and the ability to scale out using Rocksdb-Cloud [17] make it a good fit for our mapping service.

Since the mapping service is a critical component, we need to make sure that the mapping table is resilient. Deploying a single large cluster to support all data requests would work, but it difficult to manage. Instead, we decide that the mapping data is sharded within each data node. There is no need to share the across data nodes. To take advantage of this property, we could simply deploy a simple relational database on each data node. Option 2 lists the Object_mapping table in a SQLite database schema for each data node.

Updated data persistence flow

Since we have made some changes to the data node, let's revisit how to save an object in the data node (Figure 3).

1. The API service sends a request to save a new object named Object-1
2. The data node uses the quorum to determine the primary replica. Node-2 is the primary.
3. A new record of "object-1" is inserted into the object_mapping table.
4. The data node sends the object-1 to Node-2 for replication.

Figure 10. Updated data persistence flow

Durability

Data reliability is extremely important for data storage systems. How can we create a storage system that offers six nines of durability? And it takes some time for data permanently corrupted and data needs to be put back together.

Hardware failure and failure domain

Hard drive failures are inevitable no matter which models we use. Some storage media may have better durability than others, but we cannot rely on a single hard drive to achieve our durability objective. A proven way to increase durability is to replicate data to multiple hard drives. If one hard drive fails, we can still access the data from other hard drives. Replication is a common technique.

Data sources like optimizing hard drive failure and the single-disk failure rate (AFR) provide valuable insights. The Backblaze Drive Stats Q4 2022 report shows that the average AFR for hard drives is 1.35%. This number likely represents the AFR of a single hard drive. Assuming 5 copies of data goes up to 0.0018225 = 0.00006075 = 0.006075% = 0.6075 = 0.6075 percent. For more sophisticated calculations, please read [18].

For a complete durability evaluation, we also need to consider the impacts of different failure domains. A failure domain is a physical or logical unit that experiences a single point of failure. For example, a hard drive is a failure domain because if the hard drive fails, we lose all data on the hard drive. A server is a failure domain because if the server fails, all data on the server is inaccessible. A rack is a failure domain because if a rack loses power, all the servers in a rack and a rack failure domain. A modern server always comprises the motherboard, processors, power supply, NIC/HBA, and fans. Replication/erasure coding is used to mitigate hardware failures.

Since a good example of a large-scale failure domain outside. Typically, data centers divide infrastructure into areas starting with regions and availability zones. Regions are usually a large geographical area, such as Eastern US, Western US, and Central US. Each region has several availability zones. Each availability zone is a failure domain. Regions are isolated from each other. Data in a region is stored only in the region. Some regions have several availability zones, such as Google-scale power outages, cooling system failures, natural disasters, etc.

Figure 11. Data node failure domain

Erasure coding

Making three full copies of data gives us roughly 6 nines of data durability. Are there other options to further increase durability? Yes, erasure coding is a good option. Erasure coding (EC) works with algorithms efficiently to encode data into chunks and store them on different servers and create parities for redundancy. In the event of failures, we can use chunks data and parities to reconstruct the data. Let's take a look at a concrete example for 2-1 erasure coding as shown in Figure 12.

Data Node
Data Node
Data Node

1. The garbage collector copies objects from "/data/b" to a new file named "/data/d". Note the garbage collector skips "Object 2" and "Object 5" because the delete flag is set to true for both of them.
2. After all objects are copied, the garbage collector updates the object_mapping table. For example, the obj_id and object_size fields of "Object 3" remain the same, but file_name and start_offset are updated to reflect its new location. To ensure data consistency, it's a good idea to wrap the update operations to file_name and start_offset in a database transaction.

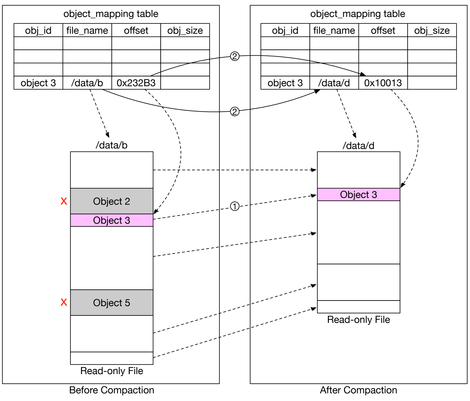


Figure 26 Compaction

As we can see from Figure 26, the size of the new file after compaction is smaller than the old file. To avoid creating a lot of small files, the garbage collector usually waits until there are a large number of read-only files to compact, and the compaction process appends objects from many read-only files into a few large new files.

Step 4 - Wrap Up

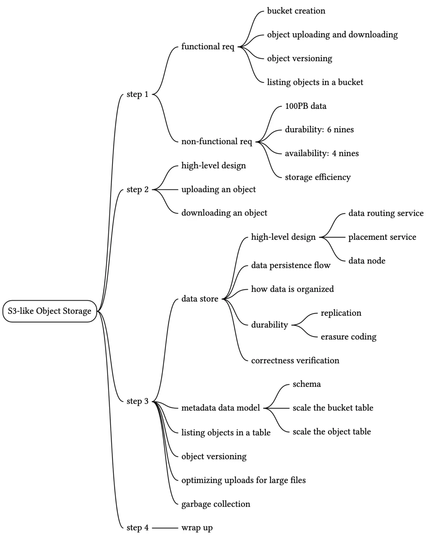
In this chapter, we described the high-level design for S3-like object storage. We compared the differences between block storage, file storage, and object storage.

The focus of this interview is on the design of object storage, so we listed how the uploading, downloading, listing objects in a bucket, and versioning of objects are typically done in object storage.

Then we dived deeper into the design. Object storage is composed of a data store and a metadata store. We explained how the data is persisted into the data store and discussed two methods for increasing reliability and durability: replication and erasure coding. For the metadata store, we explained how the multipart upload is executed and how to design the database schema to support typical use cases. Lastly, we explained how to shard the metadata store to support even larger data volume.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Chapter Summary



Reference Material

[1] Fibre channel: https://en.wikipedia.org/wiki/Fibre_Channel

[2] iSCSI: https://en.wikipedia.org/wiki/ISCSI

[3] Server Message Block: https://en.wikipedia.org/wiki/Server_Message_Block

[4] Network File System: https://en.wikipedia.org/wiki/Network_File_System

[5] Amazon S3 Strong Consistency: https://aws.amazon.com/s3/consistency/

[6] Serial Attached SCSI: https://en.wikipedia.org/wiki/Serial_Attached_SCSI

[7] AWS CLI ls command: https://docs.aws.amazon.com/cli/latest/reference/s3/ls.html

[8] Amazon S3 Service Level Agreement: https://aws.amazon.com/s3/sla/

[9] Ambry: LinkedIn's Scalable Geo-Distributed Object Store: https://assured-cloud-computing.illinois.edu/files/2014/03/Ambry-LinkedIns-Scalable-GeoDistributed-Object-Store.pdf

[10] inode: https://en.wikipedia.org/wiki/Inode

[11] Ceph's Rados Gateway: https://docs.ceph.com/en/pacific/radosgw/index.html

[12] grpc: https://grpc.io/

[13] Paxos: https://en.wikipedia.org/wiki/Paxos_(computer_science)

[14] Raft: https://raft.github.io/

[15] Consistent hashing: https://www.toptal.com/big-data/consistent-hashing

[16] RocksDB: https://github.com/facebook/rocksdb

[17] SSTable: https://www.igvita.com/2012/02/06/sstable-and-log-structured-storage-leveldb/

[18] B+ tree: https://en.wikipedia.org/wiki/B%2B_tree

[19] SQLite: https://www.sqlite.org/index.html

[20] Data Durability Calculation: https://www.backblaze.com/blog/cloud-storage-durability/

[21] Rack: https://en.wikipedia.org/wiki/19-inch_rack

[22] Erasure Coding: https://en.wikipedia.org/wiki/Erasure_code

[23] Reed–Solomon error correction: https://en.wikipedia.org/wiki/Reed%E2%80%93Solomon_error_correction

[24] Erasure Coding Demystified: https://www.youtube.com/watch?v=Q5kVuM7zEUI

[25] Checksum:https://en.wikipedia.org/wiki/Checksum

[26] Md5: https://en.wikipedia.org/wiki/MD5

[27] Sha1: https://en.wikipedia.org/wiki/SHA-1

[28] Hmac: https://en.wikipedia.org/wiki/HMAC

[29] TIMEUUID: https://docs.datastax.com/en/cql-oss/3.3/cql/cql_reference/timeuuid_functions_r.html

Mark as Complete

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.